Dr. Vishwanath Karad
MIT WORLD PEACE UNIVERSITY | PUNE
TECHNOLOGY, RESEARCH, SOCIAL INNOVATION & PARTNERSHIPS

School of Computer Science and Engineering

Department of Computer Engineering and Technology (DCET)

Mini Project: Parkwise

1032211384 : Pratham Singh

Problem Statement & Proposed Solution

The problem

- Manual records make double-booking easy
- Charges can be calculated wrongly
- Hard to see which vehicles are on site

Parkwise: the solution

- One dashboard for five parking slots
- Checks for time conflicts before booking
- Calculates charges automatically
- Keeps records behind a secure sign-in





Key Features

Availability check

See free slots and the estimated charge

Slot booking

Book 1 to 5 hours, starting on a whole hour

Search

Find by registration, booking ID or slot

Check-in and check-out

Track vehicles through each status

Extend and cancel

Change bookings, with rules for each action

Dashboard and receipt

See counts and print a booking receipt



Login and User Management

Secure sign-in

Every data route needs a signed-in user

Two roles

Administrator manages users; staff handle bookings

User CRUD

Create, view, edit and delete accounts

Hashed passwords

Salted PBKDF2, never stored as text

Lockout

Five wrong passwords lock a name for a minute

Instant removal

Deleting a user ends their session at once



Process Flow Diagram



If the slot was taken after the availability check, the server refuses the booking.

System Architecture

Client: browser

- index.html
- style.css
- app.js with Fetch API
- Five local photos



HTTP

JSON

Server: Java and Spring Boot

- Embedded Tomcat
- REST API under /api
- AuthFilter and two controllers
- Validation and pricing
- JdbcTemplate SQL

SQL

SQLite database

- parking.db file
- bookings and users tables
- 11 and 6 fields



Technical Stack with Reasoning

Layer	Technology	Why it is used
Language	Java 17	Main back-end language of the project
Framework	Spring Boot 3.2.5	Starts the server and handles REST routes and JSON
Web server	Embedded Tomcat	Included in the jar, so no separate server is needed
Data access	Spring JDBC, JdbcTemplate	Runs parameterized SQL statements
Database	SQLite	Simple local database stored in one file
Front end	HTML, CSS, JavaScript	One page with no framework
Testing	JUnit 5	Tests the controller with in-memory SQLite
Build	Maven	Manages dependencies and builds the runnable jar

Front End: HTML, CSS and JavaScript

HTML and CSS

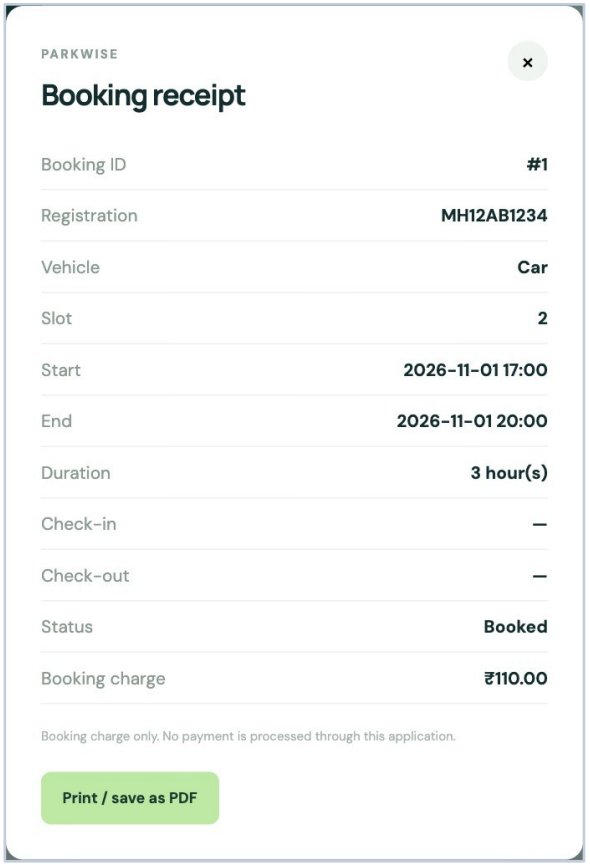
- index.html holds the one page
- style.css handles the layout
- Five photos are stored locally

Page areas

- Dashboard cards
- Booking form and rates
- Records, users and receipt

Main JavaScript functions

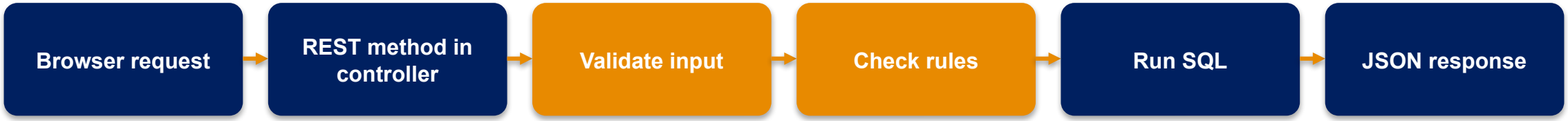
- **api():** every server request
- **loadBookings():** builds the table
- **loadStats():** fills the dashboard
- **refreshAll():** reloads both
- **showReceipt():** opens the receipt
- **escapeHtml():** keeps page text safe



Receipt dialog

Back End: Java and Spring Boot

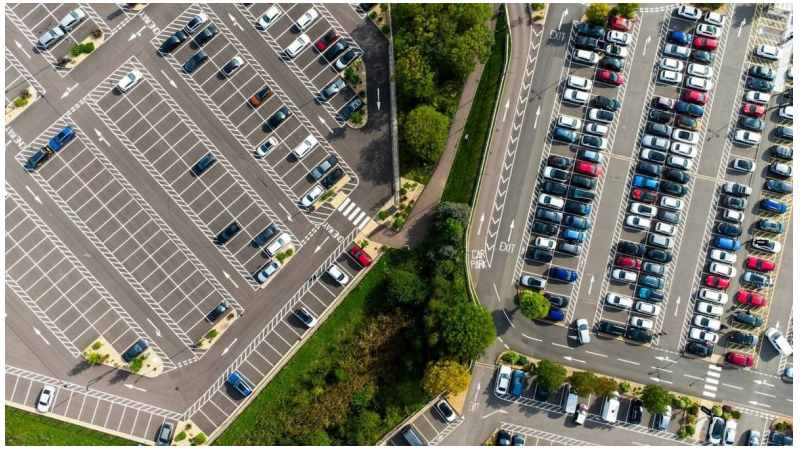
How a request is handled



Method	Endpoint	Purpose	Java method
GET	/api/availability	Check slots and estimate price	availability()
POST	/api/bookings	Create a booking	book()
GET	/api/bookings	List or search bookings	bookings()
GET	/api/bookings/{id}	Get one booking	booking()
POST	/api/bookings/{id}/{action}	Cancel, check in or check out	action()
POST	/api/bookings/{id}/extend	Extend a booking	extend()
GET	/api/statistics	Dashboard totals	statistics()

Database: bookings and users Tables

Field	Type	Purpose
id	INTEGER	Unique booking number
car_number	TEXT	Vehicle registration
slot	INTEGER	Slot 1 to 5
start_time, end_time	TEXT	Reserved period
hours	INTEGER	Total hours, 1 to 5
price	REAL	Booking charge
status	TEXT	Current booking state
vehicle_type	TEXT	CAR or TWO_WHEELER
checked_in_at	TEXT	Check-in time
checked_out_at	TEXT	Check-out time



Design

- bookings: the 11 fields shown
- users: 6 fields, password hashed
- Booking records are never deleted

Workflow: Pricing



Car

Peak: Rs. 50 per hour

Off-peak: Rs. 30 per hour



Two-wheeler

Peak: Rs. 30 per hour

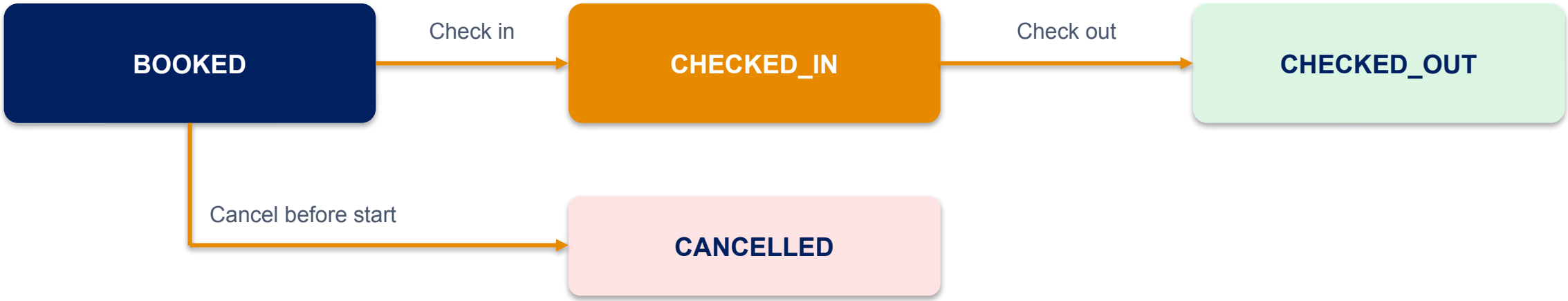
Off-peak: Rs. 20 per hour

Rules

- Peak is 09:00 up to 18:00
- Each hour uses the rate when it starts
- 10% off for 4 or 5 hours
- No payment is collected

Example, confirmed by testing	Hourly rates	Discount	Charge
Car, 08:00, 2 hours	30 + 50	None	Rs. 80
Two-wheeler, 17:00, 4 hours	30 + 20 + 20 + 20	10%	Rs. 81
Car, 16:00, 5 hours	50 + 50 + 30 + 30 + 30	10%	Rs. 171

Workflow: Check-in, Check-out, Extension



Check-in

- Any time after booking
- Slot must not be occupied

Check-out

- Frees the slot immediately
- The charge does not change

Extension

- Total of 5 hours at most
- Added time must be free

Challenges Faced during development if Any

Challenge	How the code solves it
Preventing overlapping bookings	available() counts conflicting bookings for the same slot
Slot taken after the availability check	book() checks again before saving and returns 409
Vehicle staying past its end time	A checked-in overstay blocks the slot until check-out
Price after an extension	The whole charge is recalculated from the original start
Storing passwords safely	Only a salted PBKDF2 hash is stored, never the password
Unsafe text in the page	escapeHtml() shows typed HTML as plain text

Limitation and scope of the project with future projection extension

Current limitations

- No online payment
- Five fixed slots, shared by all vehicles
- One computer, over plain HTTP
- No refund or late fee
- No-shows stay BOOKED

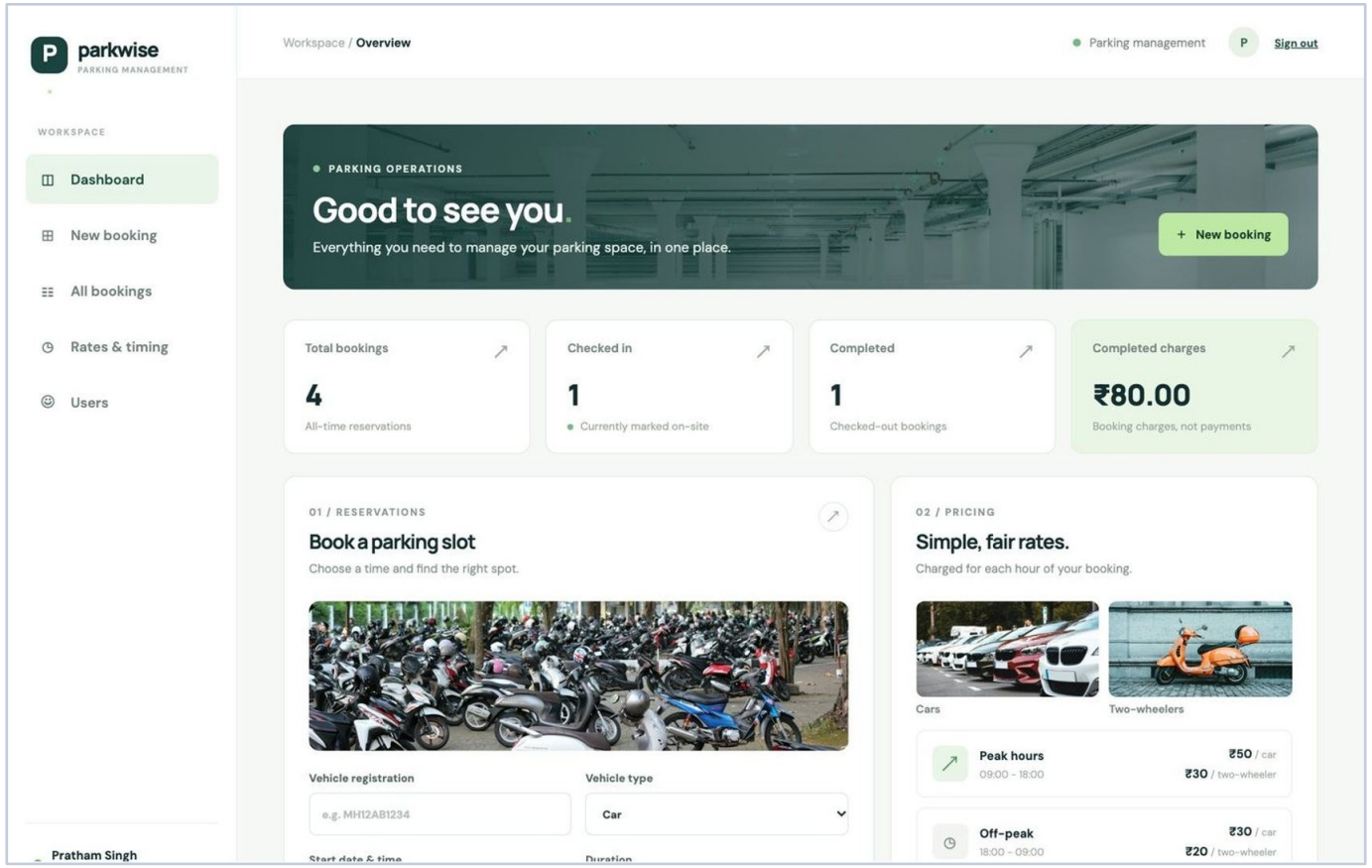
Future scope

- Add password reset by email
- Move rates and slots into settings
- Split the controller into layers
- Add database constraints and indexes
- Handle no-shows automatically
- Add online payment





Working Demo: Dashboard

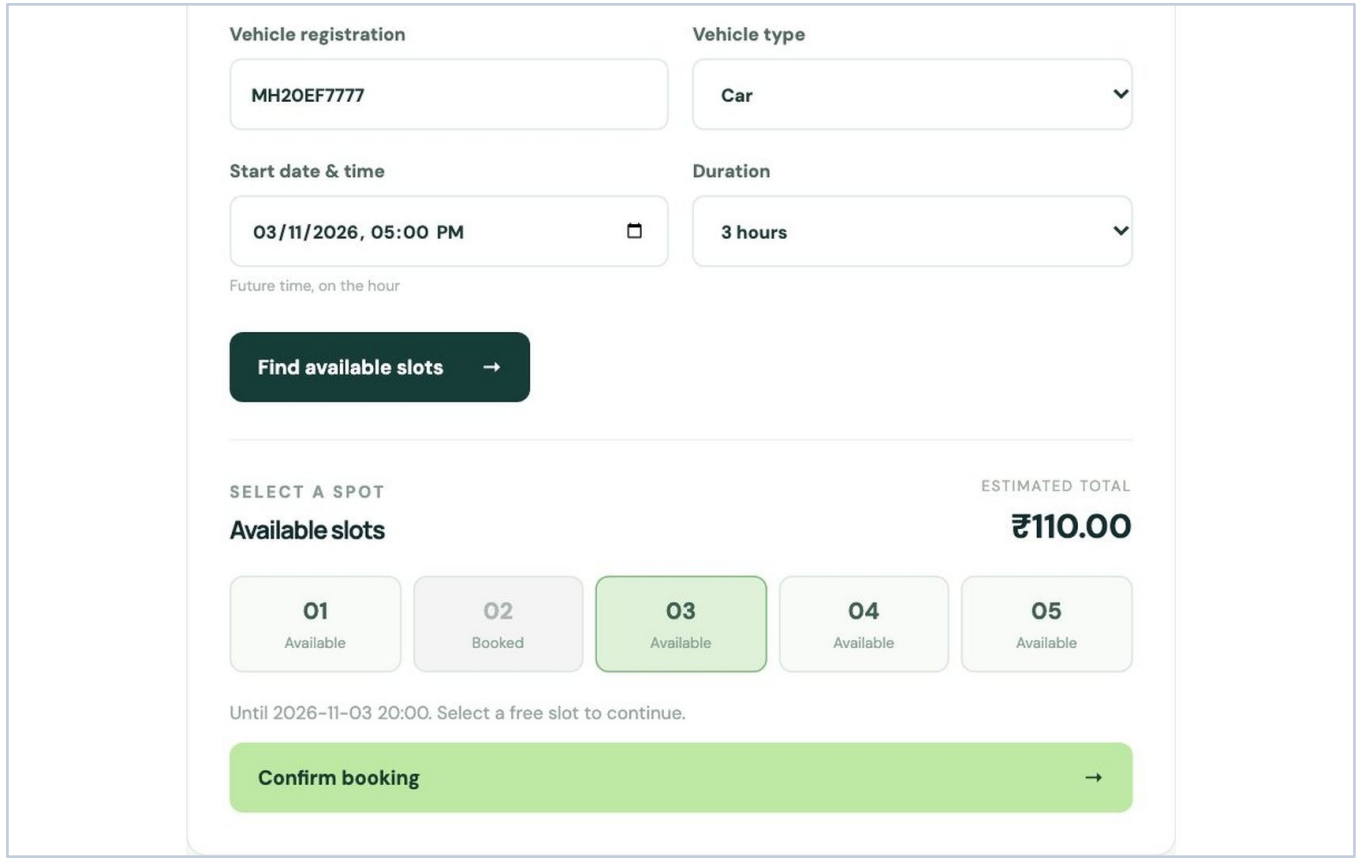


Dashboard with counts, booking form and rates

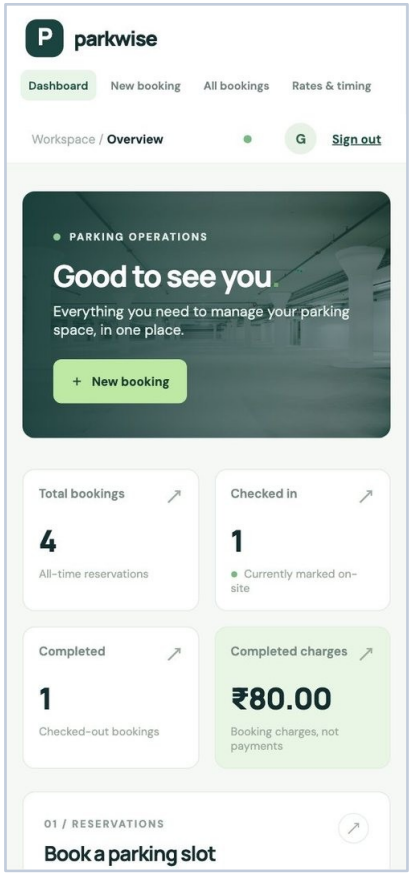
Dashboard shows

- Total bookings
- Vehicles checked in
- Completed bookings
- Completed charges
- Booking form
- Hourly rates

Working Demo: Booking



Choosing a free slot, with the estimated total

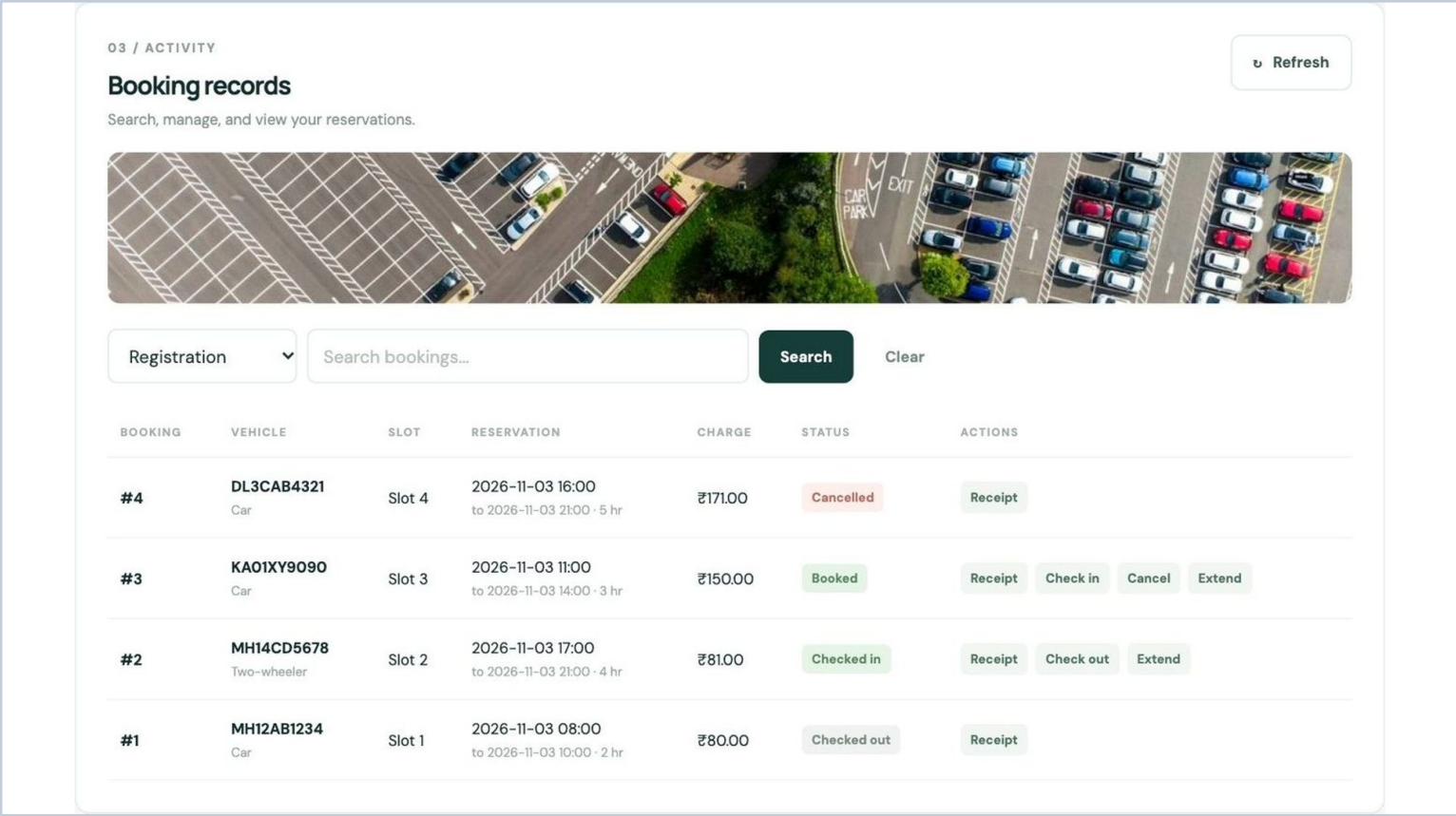


Phone view

Steps

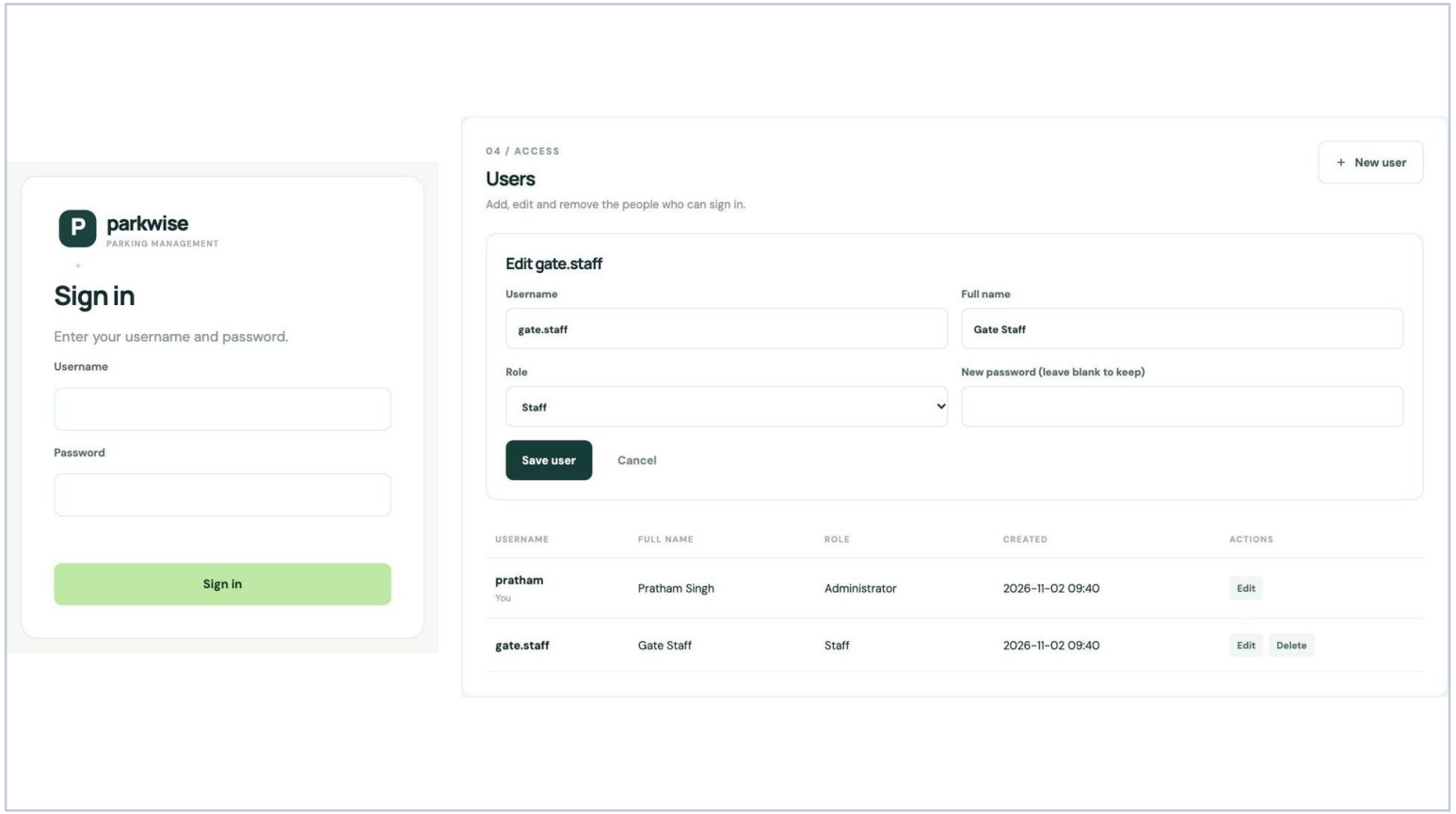
- Fill the form
- Find slots
- Choose a slot
- Confirm

Working Demo: Booking Records



Search bar, status badges and the actions allowed for each booking

Working Demo: Sign-in and Users



Sign-in screen, and the Users page with its edit form

Testing and Results

77
Login and access checks

94
Booking API checks

34
Screen checks

0
Final failures

Confirmed working

- Overlapping bookings are refused
- Prices match the worked examples
- Check-in, check-out and extension rules
- Login, roles and user CRUD work
- Other websites are refused

Not tested

- Many users at the same time
- Large numbers of bookings
- Other browsers
- JUnit on the final build
- Maven build commands

Q/A



Thank you

Photos from Unsplash: Claudio Schwarz, Benjamin Cheng, Sara Kurfess and Deski Jayantoro